



ECOLE
POLYTECHNIQUE
DE BRUXELLES

2025–2026

Computing Project

PIG: GPU Redesign of a Plenoptic Imaging Pipeline and Extension to Multi-View Reconstruction

Niccolò Benedetto

Enrollment number 626872

Prof. Daniele Bonatto

PhD Ferreira Ribeiro Brenno

June 2026



Contents

1	Background Context and Motivation	1
1.1	Integral Imaging Fundamentals	1
1.2	Existing Software	2
1.3	Project Objectives	4
2	GPU Pipeline Redesign	6
2.1	Pipeline Redesign Strategy	6
2.2	GPU Point Cloud Generation	6
2.3	GPU Post-Processing	7
2.4	Experimental Validations and Results	8

Abstract

This report presents the redesign of an existing CPU plenoptic image generation (PIG) pipeline toward a GPU implementation using CUDA. This work focuses on the acceleration about the most computationally expensive stages of the existing pipeline, and in the same time aims to preserve the rendering correctness. In addition, the new GPU pipeline is extended to support multi-view datasets (RGB + depth) by means GPU-based point cloud registration and merging. Experimental results demonstrate significant runtime improvements and validate the geometric consistency and efficiency of the proposed multi-view pipeline regarding the single-view disocclusion problems.

Background Context and Motivation

1.1 Integral Imaging Fundamentals

Integral Imaging (InI) is a glasses-free three-dimensional visualization technique based on light-field reconstruction through a Microlens Array (MLA). A plenoptic image, composed of a grid of microimages, is displayed behind the MLA, allowing each microlens to project a slightly different view of the scene toward the observer. This enables depth perception and motion parallax without requiring wearable devices. [Fer+26]

The display geometry is mainly defined by the MLA pitch D , focal length f , and the gap B between the MLA and the display panel. These parameters determine the reconstruction properties of the system. The following figure (adapted from the figure 2 in [Fer+26]) explains better the setup of the Integral Imaging.



Figure 1.1: Geometric model of an Integral Imaging display system. Adapted from [Fer+26].

The reconstructed image is focused around the Center Depth Plane (CDP), whose position is derived from the thin lens equation:

$$l = \frac{Bf}{B - f} \quad (1.1)$$

The existing software adopted uses a square MLA configuration with a pitch of 1.6056 mm, focal length of 8.028 mm, and a display–MLA gap configured for real-mode operation.

1.2 Existing Software

The Plenoptic Image Generator (PIG) software converts RGB+D data into a plenoptic image suitable for Integral Imaging displays. Basically a plenoptic image is 2D image composed by many microimages, each of these associated to a microlens. The magic happens when the plenoptic image is shown behind the MLA: each microlens projects their own microimage observed from a viewpoint slightly different onto the space, the light field is reconstructed and the observer perceives 3D effects.

Therefore at the end of the day PIG is able to convert data RGB+D from the initial scene, into microimages that are supposed to be seen through each microlens. Technically speaking PIG is entirely CPU-based and organized into 4 stages: Pre Processing, Point Cloud Reconstruction, Plenoptic Rendering and Post Processing.

The Pre-Processing stage applies super-resolution upscaling to RGB and depth images in order to densify the reconstructed point cloud. In fact a point cloud generated by only one RGB+D is sparse so the paper contributors have shown “crack artifacts” (holes) occur.

The Point Cloud Reconstruction stage converts RGD+D data into a cloud of 3D points by taking into account the intrinsic camera parameters. Assuming a pinhole camera model, each pixel (x_i, y_i) associated with depth Z_i is backprojected into 3D space according to:

$$(X_i, Y_i, Z_i)^T = \left(\frac{(x_i - p_x)Z_i}{f_c}, \frac{(y_i - p_y)Z_i}{f_c}, Z_i \right)^T \quad (1.2)$$

where f_c denotes the camera focal length and (p_x, p_y) the principal point coordinates.

Now with the Plenoptic Rendering stage we need to figure out for each 3D point from the previous stage which microlens are able to see it and where such point appears inside each microimage.

They used the center coordinates of the center of each microlens together with its angular Field-of-View (see equation 8 in [Fer+26]). Once the microlens has been found the point is projected into the relative microimage accordingly to

$$\begin{aligned} x'_{ij} &= (X_i - m_{jx}) \frac{B}{Z_i} + m_{jx} \\ y'_{ij} &= (Y_i - m_{jy}) \frac{B}{Z_i} + m_{jy} \end{aligned} \quad (1.3)$$

where m_{jx} and m_{jy} are the center coordinates of the microlens. However here they observed as well that when $Z_i \rightarrow 0$ the projection

$$\frac{B}{Z_i}$$

diverges. In other words the points near to the MLA explode out of the microimage causing artifacts. Contributors resolved this problem by filtering in the point cloud only the points above a certain depth threshold.

The last stage is about filling the remaining “crack artifacts”. Even if they applied an up-scaling via super-resolution some pixels remain empty due to the discrete nature of the point cloud. In order to address such inconvenience, once identified invalid pixel by using $\alpha=0$, they applied local median filter and copied only filtered values into holes. Here the local median filtering acts pretty well since typically crack pixels are local outliers and such filter reconstruct the missing value by preserving edges.

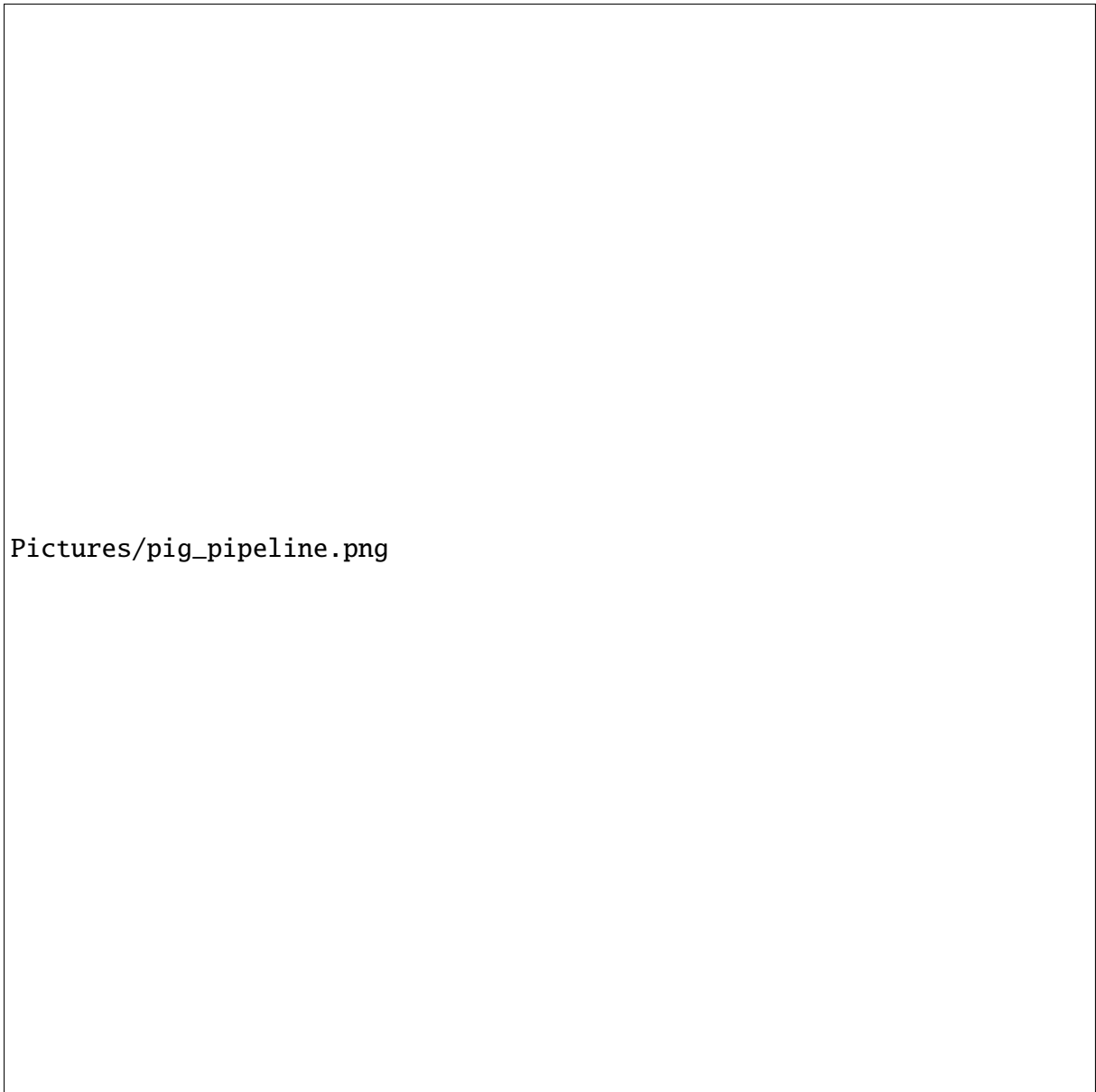


Figure 1.2: Overview of the original PIG rendering pipeline. Adapted from [Fer+26].

1.3 Project Objectives

The existing CPU-based pipeline works but suffers from runtime performances, due to its sequential nature. Stages like Point Cloud Reconstruction and Post Processing involve large amount of pixel-wise and point-wise operations making them computationally expensive. Moreover contributors observed disocclusion artifacts in the plenoptic image generated. This because of the originate scene comes from a single RGB+D viewpoint and in such scenario regions occluded cannot be recovered during the rendering.

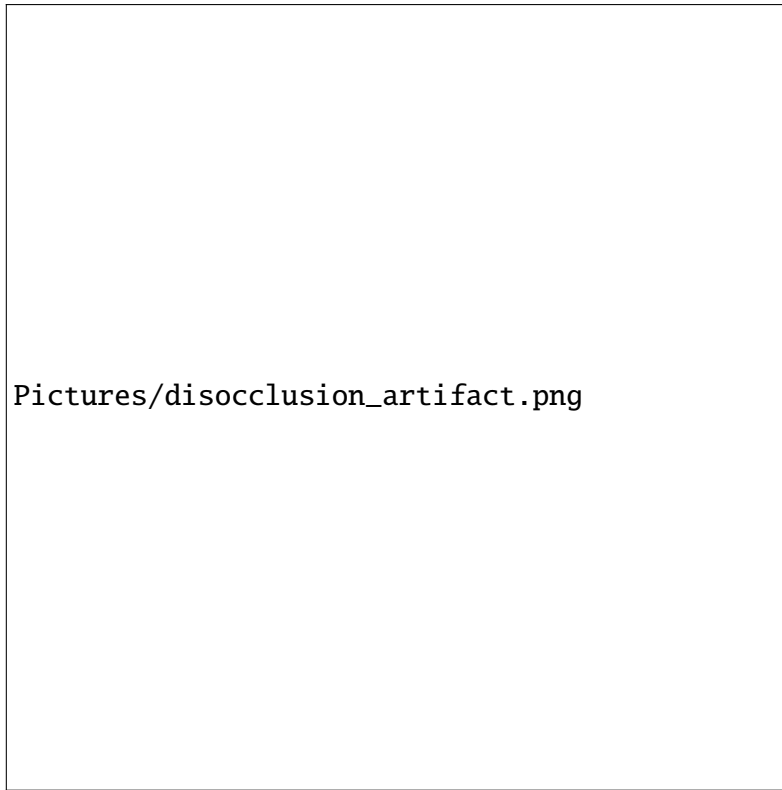


Figure 1.3: Example of crack and disocclusion artifacts generated by single-view point-cloud rendering. Adapted from [Fer+26].

These limitations motivate naturally the project objectives of this work. The first one is to redesign the most time computationally expensive stages of the original pipeline into a GPU execution while preserving rendering correctness and visual consistency. The second objective is to extend the new pipeline by implementing a new multi-view reconstruction pipeline by taking into account datasets with multiple RGB+D data.

GPU Pipeline Redesign

2.1 Pipeline Redesign Strategy

The original PIG software already exploited GPU acceleration during the plenoptic rendering stage, while the remaining stages were entirely executed on CPU. The original CPU pipeline outcome result reveals that the main computational bottlenecks were concentrated in the post-processing and point cloud generation stages. On the other hand the pre-processing stage takes only $\approx 30ms$, therefore the pipeline was redesigned to bring the remaining two most expensive stage on GPU.

(HERE FIGURE OF CPU TERMINAL RESULT)

We didn't discard the old pipeline but we introduced a boolean configuration flag in order to switch dinamically between these two and compare effectively the benchmarks.

2.2 GPU Point Cloud Generation

The point cloud generation stage converts RGB and depth images into a 3D point cloud representation suitable for plenoptic rendering. The original CPU implementation consisted of three sequential steps: Point cloud initialization $\rightarrow$ 2D-to-3D reprojection $\rightarrow$ Point cloud normalization and display

Among these stages, only the last two were migrated to GPU. Point cloud initialization was intentionally kept on CPU because profiling showed that its execution time was already way to negligible compared to the remaining stages.

The reprojection step takes as input an independet generated point cloud of the type (X,Y, Z, color) of the initialization step, still without taking into account any coordinates system. Therefore it applies the classical pinhole camera model (1.2). The GPU programming choices here basically plan to associated each pixel to one CUDA thread through a single kernel in order to configure the parallel computation. Nevertheless we early carefully dealt with the fact that before reprojecting we need to check if the current pixel is valid or not ($Z \rightarrow 0$). This problem

motivates the use of additional kernel that works as validity mask.

The CPU point cloud generation stage address the filtering by means in-place compaction ables to maintain the final data arrays without holes: scan every point of the cloud, if invalid skip it, otherwise keep it and increment a counter. Such approach would yield race conditions considering milions of threads in GPU. Therefore the new dedicated validity kernel works in a different way. Each CUDA thread processes one depth sample independently and generates a binary validity mask:

$$mask_i = \begin{cases} 1 & \text{if } Z_i < Z_{bg} \\ 0 & \text{otherwise} \end{cases} \quad (2.1)$$

where Z_{bg} represents the background depth threshold.

After masking, valid points become sparse inside the original image data arrays. We need a compact contiguous representation as CPU strategy ensures before generating the final point cloud. Surprisingly CUDA offers already a built-in method to solve this problem efficiently. In particular by exploiting NVIDIA CUB primitives we have access to a method `cub::DeviceScan::InclusiveSum()` capable to optimize prefix-sum operations, eliminating the need for atomic operations. Given a binary validity mask:

$$[1, 0, 1, 1, 0, 1] \quad (2.2)$$

the inclusive scan produces:

$$[1, 1, 2, 3, 3, 4] \quad (2.3)$$

where each value represents the compact destination index of a valid point inside the output point cloud.

Finally, after stream compaction, valid pixels are reprojected into 3D coordinates and scattered directly into compact output buffers through the last projection and scatter kernel, which simply applies final geometric normalization and positioning to the point cloud by launching one thread per valid point.

2.3 GPU Post-Processing

The post-processing, responsible for correcting visual artifacts generated during plenoptic image rendering, represented the dominant computational bottleneck due to many filtering operations applied over the plenoptic images.

The GPU redesign followed the CPU logic by focusing on two main operations: crack artifact filtering and microimage rotation.

In the crack filtering stage we proposed to apply the median filter only after decomposing the plenoptic image into independent microimage Regions of Interest (ROIs), where each ROI

corresponds to a single MLA lenslet region.

From a CUDA perspective this formulation should improve memory locality by constraining neighborhood accesses inside compact spatial regions, reducing irregular global memory traffic during filtering.

Therefore the filtering kernel was designed for a parallel computation: each CUDA thread processes one pixel and of course the filtering is applied only to crack values. This thread independence avoids synchronization operations.

At this point the problem was how to properly map the CUDA threads to the exact pixel within the ROI. We thought of decomposition as follow:

- `grid.x` and `grid.y` represent local pixel coordinates inside a microimage ROI,
- `grid.z` identifies the processed microimage region.

This organization naturally reflect the structure of the plenoptic image:

$$thread(x, y, z) \rightarrow pixel(x, y) \text{ inside microimage } z \quad (2.4)$$

We need to take into account that CUDA limits the maximum depth of `grid.z` to 65535 blocks. Since the total number of microimages may exceed this hardware constraint, the implementation introduces a batching strategy. The complete set of microimages is partitioned into multiple subsets, where each subset is processed through an independent kernel launch.

Finally the final step rotates each microimage by 180 degrees in order to match the physical optical configuration of the display system.

The microimage size is estimated from the MLA pitch and the display pixel size:

$$microimageSize = \frac{pitch_{MLA}}{pixelSize} \quad (2.5)$$

The rotation kernel follows a classic thread-pixel formulation where each CUDA thread: computes local microimage coordinates $\rightarrow$ evaluates the rotated local position $\rightarrow$ remaps the corresponding output pixel. The kernel operates directly in local microimage coordinates.

2.4 Experimental Validations and Results

The GPU redesign was experimentally evaluated by considering both rendering consistency and computational performance with respect to the original CPU pipeline, which plenoptic output is considered as ground truth and time performance as benchmark. Rendering consistency was quantitatively evaluated using the Peak Signal-to-Noise Ratio (PSNR) and the experiment was executed on the dataset `ball.json`.

We validated independently the point cloud generation and the post processing stage. The GPU post processing achieved:

Figure 2.1: Single-view RGB+D dataset used during validation experiments.

$$PSNR_{PP} \approx 33.9 \text{ dB} \quad (2.6)$$

We could not improve further this result most likely due to the neighborhood ordering or the crack-mask application that the GPU redesign introduced respect to the CPU implementation.

After that we validated the point cloud generation GPU stage. Here the PSNR dropped:

$$PSNR_{PCG} \approx 31 \text{ dB} \quad (2.7)$$

Since the post-processing stage independently achieved higher PSNR values when isolated, this suggest that the dominant rendering discrepancy originates from the GPU point cloud generation stage itself. By the way we explored refinements like evaluating float versus double precision during the computation or the stream compaction with NVIDIA CUB, in order to reproduce as closely as possible the PSNR to inf without any improvements.

Table 2.1 summarizes the execution times measured for both the original CPU pipeline and the redesigned GPU pipeline.

Table 2.1: CPU vs GPU pipeline execution times.

Stage	CPU (ms)	GPU (ms)
Pre-Processing	51.22	26.76
Point Cloud Generation	305.85	128.11
Plenoptic Rendering	422.37	36.45
Post-Processing	849.11	101.69
Total	1628.55	293.01

The redesigned GPU pipeline reduced the total execution time from approximately:

$$1628 \text{ ms} \rightarrow 293 \text{ ms} \quad (2.8)$$

corresponding to an overall speedup of approximately:

$$\frac{1628}{293} \approx 5.5\times \quad (2.9)$$

The largest improvement was observed in the post-processing stage:

$$849 \text{ ms} \rightarrow 101 \text{ ms} \quad (2.10)$$

confirming that the original CPU median-filtering operations represented the dominant computational bottleneck of the pipeline.

(HERE WOULD BE USEFUL SOME PROFILER GRAPHICS - we already know we are memory bound since in each stage we don't have complex operation but many memory accesses

but would be nice visualizing it from the profiler)

Bibliography

- [Fer+26] Ferreira, Brenno et al. (2026). “Large-Size Integral Imaging Display with Depth Image-Based Plenoptic Rendering”. In: *SPIE*.